LIBFT · PARTIE 3 · SUJET V19.3

Les listes chaînées

La structure t_list et ses neuf fonctions, avec le raisonnement derrière chaque ligne.



LE CONTEXTE

Pourquoi une liste chaînée ?

Critère	Tableau	Liste chaînée
Taille	Fixée à la création	Variable, un nœud à la fois
Insertion au début	Décale tous les éléments	Un seul pointeur à modifier
Accès au n-ième	Direct par index	Parcours depuis la tête
Mémoire	Un seul bloc contigu	Un malloc par nœud, dispersé

> Dans Libft, content est un void * : la même liste peut contenir des chaînes, des entiers ou des structures.

LA STRUCTURE

Le type t_list

```
typedef struct s_list
{
    void *content;
    struct s_list *next;
} t_list;
```

content. Un void * : l'adresse de n'importe quel type de donnée.

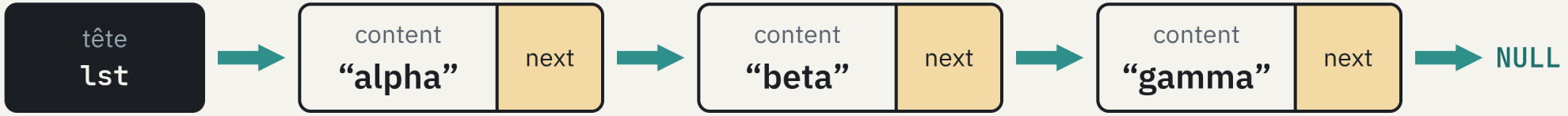
next. L'adresse du nœud suivant, ou NULL pour le dernier.

Où la déclarer. Dans libft.h, comme le demande le sujet.

Conséquence. La liste ignore ce que contient content : les opérations sur le contenu passent par des fonctions fournies par l'appelant.

LA STRUCTURE

Une liste en mémoire



UN NŒUD

Une donnée et l’adresse du suivant, soit deux pointeurs.

LA TÊTE

Un simple pointeur vers le premier nœud : c’est tout ce qu’on garde.

LA FIN

Le next du dernier nœud vaut NULL ; une liste vide est un pointeur NULL.

VUE D'ENSEMBLE

Les neuf fonctions à écrire

Fonction	Rôle	Externes
ft_lstnew	Crée un nœud avec next à NULL	malloc
ft_lstadd_front	Ajoute un nœud au début	aucune
ft_lstsize	Compte les nœuds	aucune
ft_lstlast	Retourne le dernier nœud	aucune
ft_lstadd_back	Ajoute un nœud à la fin	aucune
ft_lstdelone	Libère un nœud et son contenu	free
ft_lstclear	Libère un nœud et tous ses successeurs	free
ft_lstiter	Applique f au contenu de chaque nœud	aucune
ft_lstmap	Crée une nouvelle liste à partir de f	malloc, free

FONCTION 1 SUR 9

ft_lstnew

```c
t_list *ft_lstnew(void *content)
{
    t_list *node;

    node = malloc(sizeof(t_list));
    if (!node)
        return (NULL);
    node->content = content;
    node->next = NULL;
    return (node);
}
```

Allocation. malloc(sizeof(t_list)) réserve un nœud ; si malloc échoue, on retourne NULL.

content. L'adresse reçue est stockée telle quelle, sans copie du contenu.

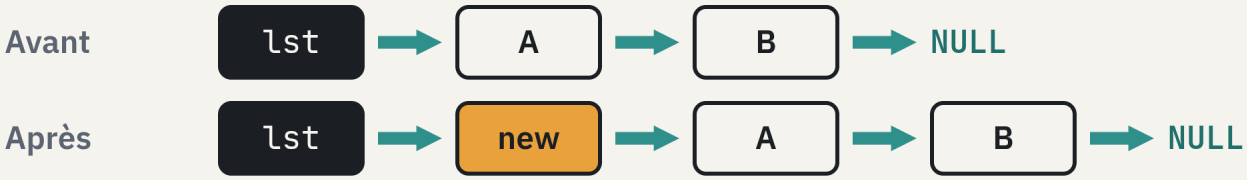
next. Initialisé à NULL : un nouveau nœud est un maillon isolé.

Cas limite. ft_lstnew(NULL) reste valide : le nœud existe, son content vaut NULL.

FONCTION 2 SUR 9

ft_lstadd_front

```c
void ft_lstadd_front(t_list **lst, t_list *new)
{
    if (!lst || !new)
        return ;
    new->next = *lst;
    *lst = new;
}
```



Deux étapes. Le nouveau nœud pointe vers l'ancienne tête, puis la tête devient le nouveau nœud.

L'ordre compte. Changer *lst d'abord ferait perdre l'ancienne liste.

L'IDÉE CLÉ

Pourquoi un double pointeur ?

Une fonction C travaille sur des copies de ses arguments. Pour changer la tête de la liste de l'appelant, il faut lui passer l'adresse de son pointeur.

```
t_list *lst;
lst = NULL;
ft_lstadd_front(&lst, ft_lstnew(content));
```

`t_list *lst`

Copie du pointeur : la fonction modifie sa copie, l'appelant ne voit rien.

`t_list **lst`

Adresse du pointeur de l'appelant : la tête peut vraiment changer.

FONCTIONS 3 ET 4 SUR 9

ft_lstsize et ft_lstlast

```c
unsigned int ft_lstsize(t_list *lst)
{
    unsigned int count;

    count = 0;
    while (lst)
    {
        count++;
        lst = lst->next;
    }
    return (count);
}
```

Parcourt la liste jusqu'à NULL en comptant chaque nœud. Une liste vide donne 0.

```c
t_list *ft_lstlast(t_list *lst)
{
    if (!lst)
        return (NULL);
    while (lst->next)
        lst = lst->next;
    return (lst);
}
```

S'arrête sur le dernier nœud (next à NULL), pas sur NULL. Une liste vide donne NULL.

FONCTION 5 SUR 9

ft_lstadd_back

```c
void ft_lstadd_back(t_list **lst, t_list *new)
{
    if (!lst || !new)
        return ;
    if (!*lst)
    {
        *lst = new;
        return ;
    }
    ft_lstlast(*lst)->next = new;
}
```

Liste vide. Si *lst vaut NULL, new devient la tête.

Sinon. ft_lstlast donne le dernier nœud, dont next pointe vers new.

Pourquoi **lst. Pour une liste vide, c'est la tête de l'appelant qui change.

Coût. Un parcours complet de la liste à chaque ajout.

FONCTION 6 SUR 9

ft_lstdelone

```c
void ft_lstdelone(t_list *lst, void (*del)(void *))
{
    if (!lst || !del)
        return ;
    del(lst→content);
    free(lst);
}
```

Contenu d'abord. del(lst->content) libère ce que le nœud référence.

Nœud ensuite. free(lst) libère le maillon lui-même.

Jamais le suivant. Seul le nœud donné disparaît, next n'est pas touché.

Après l'appel. Le pointeur reçu est invalide : l'appelant ne doit plus l'utiliser.

FONCTION 7 SUR 9

ft_lstclear

```c
void ft_lstclear(t_list **lst, void (*del)(void *))
{
    t_list *next;

    if (!lst || !del)
        return ;
    while (*lst)
    {
        next = (*lst)->next;
        ft_lstdelone(*lst, del);
        *lst = next;
    }
}
```

1. Sauvegarder next. On le lit avant toute libération.

2. Libérer le nœud. ft_lstdelone traite le contenu puis le maillon.

3. Avancer. *lst prend la valeur sauvegardée, jusqu'à NULL.

Résultat. En sortie de boucle, *lst vaut NULL : la liste est vide chez l'appelant.

FONCTION 8 SUR 9

ft_lstiter

```c
void ft_lstiter(t_list *lst, void (*f)(void *))
{
    if (!f)
        return ;
    while (lst)
    {
        f(lst→content);
        lst = lst→next;
    }
}
```

Callback. f reçoit le content de chaque nœud, pas le nœud.

Effet de bord. f modifie ou affiche le contenu ; ft_lstiter ne retourne rien.

```c
void print_str(void *s)
{
    ft_putendl_fd((char *)s, 1);
}

ft_lstiter(lst, print_str);
```

```c
t_list *ft_lstmap(t_list *lst,
        void *(*f)(void *), void (*del)(void *))
{
    t_list *new_lst;
    t_list *node;
    void *content;

    new_lst = NULL;
    while (lst)
    {
        content = f(lst->content);
        node = ft_lstnew(content);
        if (!node)
        {
            del(content);
            ft_lstclear(&new_lst, del);
            return (NULL);
        }
        ft_lstadd_back(&new_lst, node);
        lst = lst->next;
    }
    return (new_lst);
}
```

FONCTION 9 SUR 9

ft_lstmap

Nouvelle liste. L'originale reste intacte ; chaque nœud neuf reçoit f(content).

Échec d'allocation. On libère le content que f vient de créer avec del, puis toute la nouvelle liste avec ft_lstclear, et on retourne NULL.

Pourquoi del. f alloue des contenus que seul l'appelant sait libérer.

À ÉVITER

Pièges et cas limites

Piège	Conséquence	Parade
Lire lst->next après free(lst)	Use-after-free	Sauvegarder next avant de libérer
Oublier le cas *lst == NULL	Segfault	Tester !*lst avant ->next
ft_lstclear sans *lst = NULL	Double free	Mettre *lst à NULL à la fin
Échec de ft_lstmap sans nettoyage	Fuite mémoire	del(content) puis ft_lstclear
t_list * au lieu de t_list **	Tête inchangée	Passer &lst à la fonction

LA DÉFENSE

Ce que tu dois maîtriser

À SAVOIR EXPLIQUER

- Pourquoi t_list ** dans add_front, add_back et clear
- Pourquoi del est un paramètre
- La différence entre lstdelone et lstclear
- Pourquoi sauvegarder next avant de libérer
- Ce que fait lstmap si une allocation échoue

À SAVOIR RÉÉCRIRE DE TÊTE

- ft_lstnew, ft_lstadd_front et ft_lstadd_back
- ft_lstsize et ft_lstlast
- ft_lstdelone et ft_lstclear
- Une petite modification demandée pendant l'évaluation

Comprendre pourquoi chaque ligne existe compte plus que le code lui-même.